

Codebase Guide

HOW TO READ, RUN AND EXPLAIN THIS PROJECT

District Health Infrastructure Allocation System

Read this before touching the code.

It takes about fifteen minutes.

8

phases

75

files

82

tests

~2,600

lines

Run it in three commands

From a clean machine with Python 3.10+ and PostgreSQL installed:

```
make venv
```

```
make db-create
```

```
make pipeline
```

That rebuilds everything from raw data — download, reconcile, load, score, optimise, stress-test, model, map, export. Then:

```
make test      # 82 tests
```

```
make app       # opens the dashboard
```

make help lists every command. Nothing in the project is built by hand — if a file is not reproducible by a make target, it does not belong in the repo.

The eight phases

Each phase is one file in **src/** and one make target. They run in order and each writes a report to **reports/**.

Command	#	What it does
make data	0	Downloads 4 public sources, checksums them, asserts each file's shape
make reconcile	1a	Matches 705 districts across a decade of boundary changes
make load	1b	Builds the PostgreSQL schema and loads staging + core
make analytics	2	Composite Need Index as a chain of SQL views
make allocate	3	Integer program: which 25 districts, proven optimal
make sensitivity	4	10,000 re-weightings; 400 re-solves; robust vs contested
make residual	7	Predicts need from Census socioeconomics; studies residuals

Command	#	What it does
make geo	5a	Matches map polygons to districts
make snapshot	—	Exports files so the app runs without a database

What lives where

config/	The knobs. Which indicators, which weights, the budget, the constraints. Change behaviour here, not in code
sql/	Four files: schema, analytics, allocation, sensitivity. The Need Index is built here, not in Python
src/	One module per phase, plus config.py (paths), datasource.py (DB or file), snapshot.py (export)
tests/	82 tests. Read these to learn what the project promises
reports/	Seven generated reports. Never edit — they are outputs
docs/	build-log.md: every phase, every failure, every decision. The most useful file here
resources /	The PDF, deck and memo for sharing
app/	The Streamlit dashboard
data/	raw (downloaded, checksummed) - interim - processed - snapshot (committed)

Read the code in this order

Do not start at the top of src/. Start with what the project promises, then how it keeps the promise.

1. config/indicators.yml

The seven indicators, their direction, the weights. The whole methodology in one readable file.

2. sql/02_analytics.sql

How a district gets its score. Four views, each commented with why.

3. src/phase3_allocate.py

The optimiser, and three baselines it is compared against.

4. docs/build-log.md

What broke and what replaced it. Read this twice.

5. tests/test_allocation.py

The claims Phase 3 is allowed to make, written as assertions.

Five things to understand before an interview

If you can explain these five, you can defend the project.

Why percentile ranks, not raw values

Institutional births spans 20-100; stunting spans 6-60. Averaging raw numbers lets the widest-spread indicator dominate by accident while the weights claim all seven are equal. Ranking puts them on one scale, so the weights are the only thing that decides influence.

Why fuzzy matching nearly broke the project

Districts created after 2011 differ from an existing district by one directional word. The matcher scored Delhi's South East against North East at 80 — inside any sane threshold. Each such match would have given a district its sibling's population and looked perfect. Fixed with a guard that refuses a match when directional words disagree.

Why the supply adjustment was dropped

$\text{underservice} = (\text{need} \times \text{pop}) / (\text{pop} / \text{norm}) = \text{need} \times \text{norm}$. The population cancels exactly, so a supply-adjusted score carries no supply information at all. A database view proves it on all 696 districts.

Why optimisation did not beat sorting

A correct greedy reaches the same optimum, because the constraints form a partition matroid. The ILP earns its place by PROVING optimality and by detecting impossible configurations. An earlier 13% premium was measured against a badly sorted baseline and was withdrawn.

Why the first objective was unfair

It selected 25 plains districts and zero hilly or tribal ones, because a hilly district needed a 67% higher need score to compete. That inverts the IPHS standard, whose norm is lower in hills precisely because they are harder to reach.

Where every number comes from

Claim	Produced by
705 districts matched, 100%	make reconcile - reports/reconciliation_report.md
Population conserved to 7 people	tests/test_load.py
25 districts, 11 states, 6 regions	make allocate - reports/allocation_report.md
11 of 25 robust	make sensitivity - reports/sensitivity_report.md
R-squared 0.657	make residual - reports/residual_report.md
82 tests passing	make test

No figure in any document was typed by hand. If a number is questioned, re-run its command and read the report.

If something breaks

Tests fail after a change	Read the assertion message. Each test says in its docstring why it exists
core.param is missing a key	Run make load. The schema was rebuilt without reloading parameters

App shows no data	Run make snapshot, or start PostgreSQL
Map tab is empty	Run make geo then make snapshot
Cloud build fails on GDAL	geopandas belongs in requirements-pipeline.txt, never requirements.txt